

Capital.com Async Trading Platform – Simple Guide

Generated by Claude Code on 2026-05-09

Simple Documentation – Capital.com Trading Platform

What this thing does

It is a tiny computer program that talks to Capital.com. It watches prices, decides when to buy or sell, and keeps everything safe.

Big ideas (but in simple words)

* **Safety first** – if something goes wrong, the program stops trading but still listens. * **Broker is the boss** – the real numbers (what you own) come from Capital.com, not from our notebook. * **Events are like messages** – price comes in, we make a "price`█`updated" message, then other parts listen.

The puzzle pieces

| Piece | What it does | |-----|-----| | **broker** | talks to Capital.com (login, websockets, orders) | | **market** | turns raw price ticks into candles and signals | | **risk** | checks if an order is allowed | | **services** | glues everything together – start`█`up, reconciliation, order flow | | **db** | saves positions, orders, candles | | **core** | settings, logging, shared state | | **api** | simple web`█`pages you can open in a browser |

How the program starts

1. The web server (FastAPI) starts. 2. `StartupService` runs: * reads the `.env` file (your secret keys) * makes a session with the broker * pulls current positions and orders from Capital.com and writes them into the local database * opens a WebSocket so future price updates arrive * finally says "All good – you can trade now". 3. A background task called `MarketStreamer` keeps reading price messages from the queue and creates `*events*`.

Where the code lives (file names)

* `app/core/config.py` – where we keep the secret keys and settings. * `app/broker/client.py` – tiny helper that calls the Capital.com REST API. * `app/broker/websocket.py` – listens to the live price stream. * `app/market/streaming.py` – takes raw ticks, makes events. * `app/risk/manager.py` – decides if an order is allowed. * `app/services/order_workflow_service.py` – creates an order in the DB and emits an event. * `app/services/reconciliation_service.py` – makes sure the local DB matches what the broker says. * `app/api/routes.py` – a few web pages: `/health`, `/positions`. * `app/main.py` – the entry point that starts everything.

Running it (simple version)

1. **Make a virtual box** – `python -m venv .venv` and `source .venv/bin/activate`. 2. **Put the secret words** – copy `.env.example` to `.env` and write your Capital.com user name, password and a database URL. 3. **Install the tools** – `pip install -r requirements.txt`. 4. **Start the server** – `uvicorn app.main:app --host 0.0.0.0 --port 8000`. 5. Open a browser and go to `http://localhost:8000/api/health` – you should see `*healthy*`.

What you see on the screen

* **Logs** – nice JSON lines that tell you what is happening. * `/api/health` – a tiny page that says the program is alive. * `/api/positions` – a list of what you own (empty at first).

Next steps (if you want to play more)

* Replace the fake URLs in `broker/client.py` with the real Capital.com endpoints. * Write a function that listens for `OrderSubmitted` events and actually sends the order to the broker. * Add more web pages to create orders from a browser. * Put a little picture on the screen that shows the latest price.

Written in simple language so even a 5-year-old could follow along.

How to run the tiny trading program (for kids)

1. Get ready

* Get a computer that has Python 3.11 or newer. * Open a terminal (a black window where you type commands).

2. Make a safe playground

```
bash
python -m venv .env      # makes a little box for the program
source .env/bin/activate # puts you inside the box
```

You will see ``.env`` at the start of your prompt – that means you are inside the box.

3. Tell the program its secrets

```
bash
cp .env.example .env      # copy a template file
nano .env                 # open it and type:
# example (replace with your real stuff)
CAPITAL_USERNAME=your_name
CAPITAL_PASSWORD=your_pass
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost/trading
```

Save and close the file (``Ctrl+X``, then ``Y``).

4. Put the building blocks inside the box

```
bash
pip install -r requirements.txt
```

The computer will download all the pieces the program needs.

5. Turn the program on

```
bash
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

You will see something like:

```
INFO: Uvicorn running on http://0.0.0.0:8000
```

Leave this window open – the program is now listening for price data.

6. Check that it works

Open a web browser and type:

```
http://localhost:8000/api/health
```

You should see a tiny piece of JSON that says ``"status": "healthy"``.

7. Look at your positions (what you own)

In the browser go to:

```
http://localhost:8000/api/positions
```

It will show an empty list at first – that is fine.

8. What happens behind the scenes?

* The program talks to Capital.com (login, price stream). * It saves what it sees in a tiny database. * If the price goes up or down a lot, it can decide to buy or sell. * All of this is safe – if something goes wrong the program stops trading.

9. Have fun!

Now you have a tiny trading robot that watches the market. You can add more pieces: * Real order sending code. * A picture that shows the latest price. * Alerts that tell you when something important happens.

Written for a kid – short, simple steps, no scary words.